

AI / Agents

Rational agent: perceives environment + uses knowledge, history, goals to choose action maximizing performance.
Limited rationality: exact optimal action may be too expensive.
AI view: acting rationally $\neq$ acting human-like.

Uninformed Search

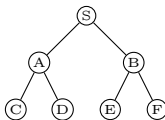
Alg	C	O	Key / Cost
BFS	Y	unit	shallowest; $T, S = O(b^d)$
UCS	Y	Y	min g ; $O(b^{G^*}/\epsilon)$
DFS	finite m	N	deepest; $T = O(b^m), S = O(bm)$
DLS	$d \leq l$	N	cutoff l ; $T = O(b^l), S = O(bl)$
IDS	Y	unit	repeated DLS; $T = O(b^d), S = O(bd)$

b : branch; d : optimal depth; m : max depth; ϵ : min step cost.
Trap: BFS optimal only for equal costs. UCS orders by total path cost. DFS may fail in infinite depth.

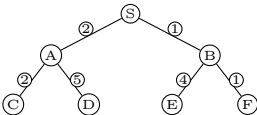
BFS / UCS / DFS Mini

BFS: FIFO fringe, append children to end. UCS: priority queue by accumulated $g(n)$. DFS: stack/LIFO, low memory.
Mini: $S \rightarrow A = 2, S \rightarrow B = 1, A \rightarrow G = 2, B \rightarrow G = 10$. UCS expands B first, but returns S-A-G cost 4.

Search Tree Mini



BFS: S, A, B, C, D, E, F . DFS L $\rightarrow$ R: S, A, C, D, B, E, F .
DLS $l = 1$: S, A, B . DLS $l = 2$: S, A, C, D, B, E, F .
IDS: run DLS 0, 1, 2, ... until goal found.



UCS: pop min path cost $g(n)$: S, B, A, F, C, E, D under shown costs.

Informed Search

Greedy best-first: minimizes $h(n)$ only. Fast if heuristic good; not optimal.
A*: minimizes total estimated path cost.

$$f(n) = g(n) + h(n)$$

g : cost so far, h : estimated remaining cost.
Admissible: $h(n) \leq h^*(n)$, never overestimates.
Consistent: $h(n) \leq c(n, n') + h(n')$.
Consistent $\Rightarrow$ admissible. Admissible $\nRightarrow$ consistent.
A* optimal if h admissible. Optimally efficient if h consistent.
Dominance: if $h_2 \geq h_1$ and both admissible, h_2 expands fewer/equal nodes.
Relaxed problem: remove constraints; relaxed solution cost gives admissible h .

A* Mini

Edges: $S \rightarrow A = 2, S \rightarrow B = 1, A \rightarrow G = 4, B \rightarrow G = 10$;
 $h(A) = 3, h(B) = 1, h(G) = 0$.
 $f(A) = 2 + 3 = 5, f(B) = 1 + 1 = 2$. **A*** expands B first, sees G cost 11, then expands A and returns S-A-G cost 6.
Trap: In **A***, goal generated $\neq$ goal selected. Return when goal is popped.

Local Search

Hill climbing: keep one current state; move to best neighbor. Not complete/optimal.
For minimization: accept child if $v(child) < v(current)$.
Problems: local optimum, plateau, ridge.
Fix: random restart; sideways moves with visit limit; macro moves; limited lookahead.
Beam search: keep best k states per level. Low memory; not complete/optimal.
Trap: Beam width k is not branching factor b . Discarded states are gone.

Simulated Annealing / GA

Simulated annealing: random neighbor; accept better always; accept worse with probability:

$$p = \exp\left(\frac{v(current) - v(next)}{T}\right)$$

for minimization. High T : exploration; low T : greedy.
Cooling too fast $\Rightarrow$ local optimum. Too slow $\Rightarrow$ inefficient.
Mini: $v(c) = 10, v(n) = 11, T = 2$. Worse move: $p = e^{-1/2} = 0.607$. If random $r = 0.4$, accept.

Genetic algorithm: population + fitness + selection + crossover + mutation.

$$P_i = \frac{f(S_i)}{\sum_j f(S_j)}$$

Selection gives uphill pressure; crossover/mutation gives exploration. Encoding matters.

Games

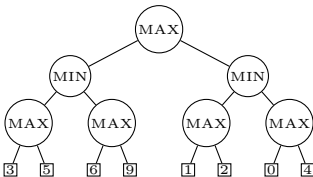
Minimax: 2-player, deterministic, perfect-info, zero-sum. MAX assumes MIN optimal.

$$V(s) = \begin{cases} U(s), & \text{terminal} \\ \max_{s'} V(s'), & \text{MAX} \\ \min_{s'} V(s'), & \text{MIN} \end{cases}$$

Time $O(b^m)$, space $O(bm)$.
Mini: MAX choices: $A \rightarrow \{3, 5\}, B \rightarrow \{2, 9\}$. MIN backs $A=3, B=2$. MAX chooses A.
Alpha-beta: same result as minimax; skips irrelevant branches. α : best lower bound for MAX. β : best upper bound for MIN.
Prune when $\alpha \geq \beta$. Best $O(bm/2)$, worst $O(b^m)$. Ordering matters.
Mini: Root MAX has left value 100, so $\alpha = 100$. At right MIN, first child 20 gives $\beta = 20$. Since $\beta \leq \alpha$, prune rest.
Expectiminimax: chance node:

$$V = \sum_i p_i V_i$$

Mini: Chance children 2,4 with $p = .5$: $V = 3$.
Game Tree Mini



Minimax: bottom-up; MAX takes max, MIN takes min. Left MIN=min(5, 9) = 5; right MIN=min(2, 4) = 2; root=max(5, 2) = 5.
 $\alpha\beta$ L $\rightarrow$ R: after left subtree, root $\alpha = 5$. In right MIN, first child gives $\beta = 2$. Since $\beta \leq \alpha$, prune remaining child (0, 4).

ML Basics

Supervised: labelled data. Classif. = discrete label; regression = numeric value.
Unsupervised: no labels. Clustering = group similar examples.
Reinforcement: feedback/reward. **Association:** co-occurring patterns.
Train set: fit model. Valid. set: tune/prune/early stop. Test set: final comparison only.

Evaluation

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}$$
$$Error = 1 - Acc$$

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

Precision: predicted positive, how often right? Recall: real positive, how many found?
Use F_1 when classes imbalanced or FP/FN both matter.
Train = fit model; validation = tune/prune/early stop; test = final unbiased comparison.
Stratification: preserve class proportions. **k-fold CV:** each fold test once; average metrics.
LOOCV: $k = n$; expensive; deterministic. **Paired t-test:** same splits; fold-wise differences.
Trap: Never tune on test set.

Clustering Concepts

Unsupervised partition: same-cluster similar, different-cluster dissimilar.
Good clustering = high cohesion + high separation.
Centroid = mean point, may not be real item. Medoid = actual central item.

$$c_i = \frac{1}{|K_i|} \sum_{x \in K_i} x$$

Agglomerative hierarchical: start singleton clusters; repeatedly merge closest clusters; update distance matrix after each merge.
Single link:

$$d(K_a, K_b) = \min_{x \in K_a, y \in K_b} d(x, y)$$

Complete link:

$$d(K_a, K_b) = \max_{x \in K_a, y \in K_b} d(x, y)$$

Average link: average pairwise distance.
Single link: chain effect. Complete link: compact clusters.
Davies-Bouldin:

$$DB = \frac{1}{k} \sum_i \max_{j \neq i} \frac{S_i + S_j}{d(c_i, c_j)}$$

Small DB = compact + far apart = good.
Mini: $K_1 = \{0, 2\}, K_2 = \{8, 10\}$. $c_1 = 1, c_2 = 9, S_1 = S_2 = 1, DB = .25$.

K-Means

Partitional, iterative, distance-based clustering.
Procedure: choose k centroids $\rightarrow$ assign to nearest $\rightarrow$ recompute centroids $\rightarrow$ repeat until stable.
Objective:

$$SSE = \sum_{i=1}^k \sum_{x \in K_i} d(x, c_i)^2$$

Time $O(tkmn)$; space $O((m+k)n)$.
Pros: simple, popular, efficient, usually fast.
Cons: need k ; init-sensitive; local minima; outliers; poor on non-spherical clusters.
Mini: Points 1, 2, 8, 9, $k = 2$, init $c_1 = 1, c_2 = 9$. Clusters $\{1, 2\}, \{8, 9\}$; new centroids 1.5, 8.5; stable.

Image Compression

1024^2 image, 2×2 blocks $\Rightarrow 512^2$ vectors, dimension 4:

$$rate = \frac{k \cdot 4 \cdot 8 + 512 \cdot 512 \log_2 k}{1024 \cdot 1024 \cdot 8}$$

NN Clustering / Hierarchical

Nearest Neighbor Clustering: incremental; process each item once.
If $d(new, K) \leq t$, join closest cluster; else new singleton. Usually single-link:

$$d(x, K) = \min_{y \in K} d(x, y)$$

Order-sensitive; threshold-sensitive; $O(n^2)$; chain effect.
Mini: $t = 2$: A starts; B dist 1 joins; C min 2 joins; D min 1 joins; E min 3 forms new cluster.
Hierarchical: dendrogram. No need k in advance.
Agglomerative = bottom-up merge. Divisive = top-down split.
Time $O(n^3)$ or $O(n^2 \log n)$; space $O(n^2)$.
Trap: Update distance matrix after each merge.

kNN

Lazy, instance-based. Training = store data.
kNN solve: compute all distances $\rightarrow$ sort ascending $\rightarrow$ take nearest $k \rightarrow$ majority vote / average.
Classif. = majority vote. Regression = average.
Distances:

$$D_E = \sqrt{\sum_i (a_i - b_i)^2}$$
$$D_M = \sum_i |a_i - b_i|$$

Normalization:

$$a_i = \frac{v_i - \min v_i}{\max v_i - \min v_i}$$

Weighted kNN:

$$w_i = \frac{1}{D(x, x_i)}, \quad \hat{y} = \sum_i w_i y_i$$

Mixed attrs: nominal match=0, mismatch=1; combine with numeric using chosen distance, e.g. $d = \sqrt{\sum(\Delta num)^2 + \sum(\Delta nom)^2}$.
Missing: worst-case / ignore depending setting.
Small $k \Rightarrow$ overfit/noisy. Large $k \Rightarrow$ smooth/biased.
Curse of dim.: distances become similar; irrelevant attrs hurt.
Mini: Training (4, L), (9, M), (8, M), (15, H), (7, M), query 5. Distances 1, 4, 3, 10, 2. 1NN=L; 3NN=L,M,M $\Rightarrow$ M.

1R

One-rule classifier using one attribute.
1R solve: for each attr A , group by value v ; each group predicts majority class; group error = minority count; total error(A) = sum group errors; choose min-error attr. Classify new example using only that attr's rule.
Numeric: sort values; candidate breakpoints where adjacent class changes:

$$breakpoint = \frac{v_i + v_{i+1}}{2}$$

Mini: Age ≤ 30 : yes=2, no=3 $\Rightarrow$ predict no, error=2. Humidity example total error = $3 + 1 = 4$.

Naive Bayes

Choose class with largest posterior score:

$$\hat{c} = \arg \max_c P(c) \prod_j P(x_j | c)$$

Assumption: attrs conditionally independent given class. Often false: attrs may remain correlated even given class; still useful if posterior ranking works.
Bayes:

$$P(H|E) = \frac{P(E|H)P(H)}{P(E)}$$

Denominator common across classes, ignore for argmax.
NB solve: for each class c , use only values in new $E = (x_1, \dots, x_d)$: $score(c) = P(c) \prod_j P(x_j | c)$. Nominal $\rightarrow$ count within class; numeric $\rightarrow$ Gaussian density $f(x_j | \mu_{jc}, \sigma_{jc})$; choose max.
Laplace:

$$P(x = v | c) = \frac{count(x = v, c) + 1}{count(c) + k}$$

m -estimate:

$$P(x = v_i | c) = \frac{n_i + m p_i}{n + m}$$

Gaussian numeric:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/(2\sigma^2)}$$

Missing new attr: omit from product. Missing train value: omit from counts/denominator.

Mini: $P(Y) = 9/14, P(sunny|Y) = 2/9, P(cool|Y) = 3/9$. Score $Y = (9/14)(2/9)(3/9) = 1/21$.

Trap: Use class-conditional probabilities, not unconditional probabilities.

Decision Trees

Choose split with highest information gain.
Entropy: $H = 0$ if pure; binary balanced $H = 1$; for k classes, $0 \leq H(S) \leq \log_2 k$, max when classes uniform.

$$H(S) = - \sum_i p_i \log_2 p_i$$

$$H(S|A) = \sum_v \frac{|S_v|}{|S|} H(S_v)$$

$$Gain(S, A) = H(S) - H(S|A)$$

IG solve: compute parent $H(S)$; split by A ; for each child S_v , count class ratio and compute $H(S_v)$; weighted child entropy; gain = parent - weighted child; choose max gain.
Trap: Branch weights $|S_v|/|S|$ are only for weighting. Child entropy uses class distribution inside branch; branch-size entropy is SplitInfo/ClassRatio.
Mini: 9 yes, 5 no: $H(S) = .940$. If $H(S|Outlook) = .693$, gain = .247.
Stop: pure; identical attrs but mixed labels $\rightarrow$ majority; empty subset $\rightarrow$ parent majority.
Overfit: train low, test high. Causes: small data, noise, too many branches.
Pre-prune: max depth / min leaf / min gain. Risk: underfit.
Post-prune: full tree + valid. test; replace subtree with leaf if valid. accuracy improves/same.
Rule pruning: tree $\rightarrow$ rules; remove harmless preconditions; sort by valid. accuracy.
Numeric attrs: sort values; candidate thresholds at class-change midpoints.
Missing values in DT gain: assign branch probabilities from non-missing examples.
Cost-sensitive split:

$$\frac{Gain(S, A)^2}{Cost(A)} \quad \text{or} \quad \frac{2Gain(S, A) - 1}{(Cost(A) + 1)^w}$$

Gain ratio:

$$SplitInfo = - \sum_i \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

$$GainRatio = \frac{Gain}{SplitInfo}$$

Gain ratio penalizes many-valued attrs.
Trap: Do not maximize remaining entropy. Maximize entropy reduction.

Perceptron

Single-layer neural classifier; linear boundary.

net = w · p + b = ∑i wi pi + b

a = step(net) = { 1, net ≥ 0; 0, net < 0

e = t - a, wnew = wold + ep, bnew = bold + e

Solve: each example in order: n = wT x + b → a = step(n) → e = t - a → update w, b immediately. Course: step(0) = 1. Decision boundary:

w · p + b = 0

Converges iff data linearly separable. Cannot solve XOR / nonlinear boundary. No bias ⇒ boundary passes origin.

Mini: w = [0, 0], b = 0, p = [0, 1], t = 0. a = 1, e = -1, w = [0, -1], b = -1.

Trap: Perceptron learning rule not guaranteed for non-linearly separable data.

MLP / Backprop

MLP with hidden layer + nonlinear differentiable activation forms nonlinear boundary. One hidden layer is universal approximator, but theorem does not tell how to find weights.

Forward: for each neuron, n = wT x + b, a = f(n). FC params = ∑l nl nl+1 + ∑l≥1 nl, where n0 is input layer and later layers have biases.

Sigmoid:

f(x) = 1 / (1 + e^-x), f'(x) = f(x)(1 - f(x))

Tanh derivative:

f'(x) = 1 - a^2

SSE:

E = 1/2 ∑i (di - oi)^2

Generic update:

Δwpq = ηδqop, wpq ← wpq + Δwpq

Output neuron:

δq = (dq - oq)f'(netq)

Sigmoid output:

δq = (dq - oq)oq(1 - oq)

Hidden neuron:

δq = f'(netq) ∑i wqi δi

Sigmoid hidden:

δq = oq(1 - oq) ∑i wqi δi

Momentum:

Δwpq(t) = ηδqop + μΔwpq(t - 1)

η: learning rate; μ ∈ [0, 1): momentum.

Backprop Procedure / Issues

Backprop solve: forward net → o; compute output δ; compute hidden δ from downstream old wδ; then update all weights. Do not round early. Stopping: error threshold; max epochs; early stop on valid. set. Input neurons: numeric directly; nominal via one-hot. Output: binary → 1 output; k classes → k outputs. Often use .9/.1 instead of 1/0 to avoid flat gradient. Hidden heuristic: roughly (ni_n + nout)/2, then tune. Too few hidden units ⇒ underfit. Too many ⇒ overfit/slow. LR too small ⇒ slow. LR too large ⇒ oscillate/diverge. Overfit causes: small data, noise, too many trainable params. Fix: early stopping, different init, adaptive LR, dropout. Local minima: GD may find local not global min. Vanishing gradient: lower layers get tiny updates. Trap: Backprop needs differentiable activation. Hidden neurons have no direct target values.

Deep / Autoencoder / CNN

Deep: multiple hidden layers; learns hierarchic features. Autoencoder: target=input, y = x; bottleneck/narrow hidden layer learns compressed representation.

CNN: image/grid data; local connectivity reduces params; weight sharing applies same learned filter across locations → feature map.

Convolution:

feature = ∑i,j xij wij

Max pooling:

pool = max(x1, ..., xr)

Pooling reduces dimension and gives shift invariance. Dropout: random deactivation during training; reduces overfit. Trap: CNN filter weights are learned and shared, not separate per pixel/location.

SVM

Maximum-margin hyperplane. SVs define boundary.

w · x + b = 0; w · x + b = ±1; margin = 2 / ||w||

Hard margin:

min 1/2 ||w||^2 s.t. yi(w · xi + b) ≥ 1

Dual / classify:

w = ∑i λi yi xi; f(z) = ∑i λi yi (xi · z) + b; ŷ = sign(f(z))

Soft margin:

0 ≤ λi ≤ C

Large C: fewer train errors, narrower margin. Small C: wider margin, more violations.

Kernel:

K(u, v) = Φ(u) · Φ(v); K(x, y) = (x · y + 1)^p; K(x, y) = e^(-||x-y||^2 / (2σ^2))

Kernel trick: compute dot product in feature space without explicit mapping. Linear separator in feature space = nonlinear boundary in original space.

Mini: w = (1, 1), z = (2, 1), b = 0: f = 3 > 0 ⇒ +1. Trap: Not all points define boundary; SVs do. SVM checks: maximizes margin; λi > 0 means support vector. Training dot products: training vectors with training vectors. Testing: support vectors with new example. Ensemble checks: Bagging works for classification and regression. Boosting often helps but does not guarantee improvement.

SVM / Perceptron / NN

Perceptron: simple, efficient, linear only. Backprop NN: nonlinear, expressive, but many params, slow, local minima, architecture/LR sensitive. SVM: margin + kernels, strong generalization, but kernel/C tuning and train cost matter. Linear SVM: optimal linear boundary in original space. Nonlinear SVM: optimal linear boundary in feature space. Multi-class SVM: decompose into binary classifiers, e.g. one-vs-one gives k(k - 1)/2 classifiers.

Ensembles

Good ensemble = individual accuracy + error diversity. Majority vote:

ŷ = mode(h1(x), ..., hM(x))

Weighted vote:

ŷ = arg maxc ∑m αm 1[hm(x) = c]

	Diversity	Train	Vote
Bag	bootstrap	parallel	equal
Boost	hard cases	seq.	weighted
RF	boot. + features	parallel	equal

Bagging reduces variance. Boosting focuses on difficult cases. RF lowers tree correlation.

Trap: Many identical classifiers do not help.

Bagging

Bootstrap aggregating. Sample n examples with replacement; train one classifier per sample; majority vote / average. About 63% of original examples appear in a bootstrap sample. Regression:

ŷ = 1/M ∑m=1M hm(x)

Useful for unstable learners e.g. DT. Easy to parallelize. Less useful for stable learners.

Mini: Original IDs 1,2,3,4,5. Bootstrap sample: 2,5,2,1,4.

Trap: Bagging samples with replacement and uses equal voting.

Boosting / AdaBoost

Sequential learners; later classifiers focus on examples earlier classifiers got wrong.

Weighted error:

εt = ∑i pi ei

ei = 1 if misclassified.

βt = εt / (1 - εt)

Correct examples:

pi ← pi βt

then normalize. Classifier vote weight:

αt = log 1 / βt

Mini: 10 examples weight .1. Classifier gets 3 wrong: ε = .3, β = .3/.7 = .429. Correct examples downweighted; wrong examples relatively upweighted.

Cons: sensitive to noisy labels/outliers; sequential not parallel. Trap: Final boosting vote is weighted, not equal.

Boosting solve: ε = sum weights of misclassified; β = ε/(1 - ε); correct examples × β, wrong unchanged; normalize all.

Random Forest

RF = bagging + random feature selection + unpruned DTs. For each tree: bootstrap sample; grow DT without pruning; at each split choose k random features; select best among k using IG; majority vote.

Rule:

k ≈ log2 m

m = total features. Performance depends on high tree strength and low correlation. Increasing k: strength ↑, correlation ↑. Pros: often better than single DT; resists overfit; faster than AdaBoost with similar accuracy. Trap: Standard RF trees are not pruned; do not use all features at every split.

Probability Basics

Conditional:

P(A|B) = P(A, B) / P(B)

Product:

P(A, B) = P(A|B)P(B) = P(B|A)P(A)

Independence:

P(A, B) = P(A)P(B)

Disjunction:

P(A ∨ B) = P(A) + P(B) - P(A ∧ B)

Bayes:

P(H|E) = P(E|H)P(H) / P(E)

Total probability:

P(X) = ∑i P(X|Yi)P(Yi)

Chain rule:

P(x1, ..., xn) = ∏i P(xi|x1, ..., xi-1)

Full joint solve: add rows where event is true. P(A): sum rows with A; P(A, B): sum rows with both; P(A|B) = P(A, B)/P(B). For scores, normalize by total sum.

Normalize:

P(X|e) = αP(X, e)

Mini: P(M) = 1/50000, P(S|M) = .5, P(S) = 1/20. P(M|S) = .5(1/50000)/(1/20) = .0002.

Trap: Do not reverse P(A|B) and P(B|A).

Bayesian Networks

BN = directed acyclic graph + CPTs.

P(x1, ..., xn) = ∏i P(xi|Parents(xi))

BN factor solve: for each node, write P(node|parents); root nodes use priors. Joint = product of all local CPT terms. Use graph, not variable names. Node cond. independent of non-descendants given parents. Markov blanket = parents + children + children's parents. Node independent of all outside blanket given blanket.

If Boolean vars have at most k parents: O(n2^k) params instead of O(2^n).

Naive Bayes = special BN: class node parent of all features.

Exact Inference

Goal: posterior such as P(X|E = e).

Enumeration:

P(X|e) = αP(X, e) = α ∑h P(X, e, h)

BN inference solve: query Q, evidence e, hidden vars h = all non-query/non-evidence vars. Compute one score per query value, sum over h, then normalize.

Variable elimination: same math, but multiply factors and sum out hidden vars one at a time, reusing intermediate factors.

Example:

P(B|j, m) = α ∑e ∑a P(B)P(e)P(a|B, e)P(j|a)P(m|a)

Trap: Do not eliminate query/evidence vars. Always normalize. Elimination order affects factor size.

Classifier Comparison

Alg	Exam summary
kNN	simple; no training; slow query; high memory; scale/irrelevant attrs hurt; append easy.
DT	interpretable; efficient; overfits; unstable; usually rebuild for updates.
NB	fast; one scan; robust; independence assumption; count update easy.
SVM	margin/kernel model; costly training; kernel/C tuning; re-train hard.
NN	nonlinear; expressive; slow; local minima; architecture/LR sensitive.
RF	accurate; robust; less interpretable; many trees raise prediction cost.

Common Exam Traps

A* returns goal when selected, not generated. BFS optimal only for equal costs. UCS uses accumulated path cost. Greedy uses h only; A* uses g + h. Admissible ≠ consistent; consistent ⇒ admissible. Precision denom. = predicted positives; recall denom. = real positives. Valid. set for tuning/pruning/early stopping; test set for final evaluation. Normalize numeric features for kNN/K-means. Naive Bayes multiplies class-conditional probabilities. DT information gain uses weighted child entropy. Backprop needs differentiable activation. Perceptron cannot solve XOR. SVM boundary defined by SVs. Bagging samples with replacement; boosting is sequential. BN inference requires normalization.